

Google Summer of Code (GSoC)

From Open Source newbie to GSoC Contributor: A Step-by-Step Guide

...

01 What are Open source Contributions?

Individuals or groups contributing to open-source projects, where source code is freely available for **use, modification**, and **distribution.**



02 Why are they Important?

1. **Improved Quality:** Collaborative review and modification lead to higher quality software.
2. **Innovation:** Drives new features, improvements, and entirely new projects.
3. **Collaboration:** Fosters community and knowledge sharing among developers.
4. **Accessibility:** Often provides free software, increasing access for all.
5. **Empowerment:** Gives users control over the software they use.

03 What can you contribute?



CODE

Writing, fixing, or improving code.



DOCUMENTATION

Creating or enhancing project documentation.



TESTING

Testing software and reporting bugs.



DESIGN

Contributing to the design of the software.



COMMUNITY

Helping other users, answering questions.

• • •

04 Git Ready to Code: Open Source Awaits!

HACKTOBER FEST

KWoC
KHARAGPUR WINTER OF CODE

MLH
MAJOR LEAGUE HACKING

Google Summer of Code

GIRLSCRIPT
Summer Of Code

Season of KDE



05 GSoC 101 - Summer of Code, Not Summer of Boredom!

Google Summer of Code or GSoC is a global program connecting students (and others 18+) with open-source organizations for 12+ weeks of contributing to real-world projects. Think paid coding internship with mentorship!



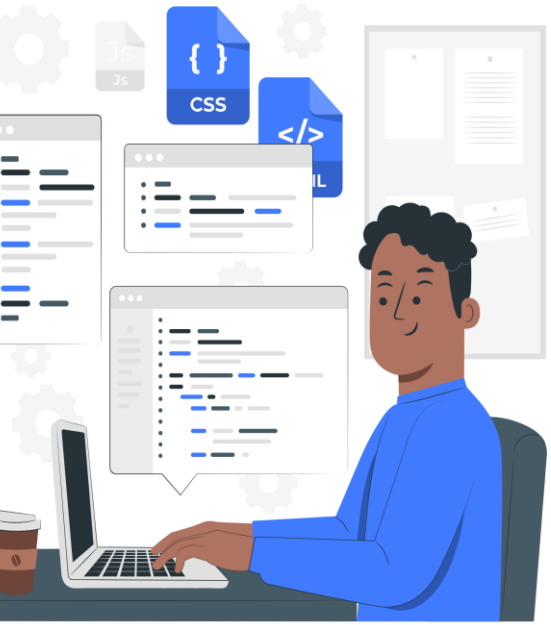
Who can apply? (Are You the Chosen One?)

If you're 18 or older and have a passion for code (or want to discover one), you're in the running! Students and non-students alike are welcome.

Why should you apply?

1. Get mentored by industry experts.
2. Build/contribute to real-world projects.
3. Earn a stipend
4. Boost your resume.

06 The Complete GSoC Timeline



Jan 27 – Feb 11	Organization Application Period
Feb 27	Organizations Announced
Mar 24 – Apr 8	Contributor Application Period
Apr 8 – Apr 29	Proposal Review Period
May 8	Accepted Projects announced
May 8 – Jun 1	Community Bonding Period
Jun 2	Coding period begins
Jul 18	Mid Evaluations
Jul 25 – Sep 1	Final submission period
Sep 1 - 8	Mentors Submit Final Evals
Nov 19	Program Ends



07 Fork Yeah! Your Roadmap to GSoC Glory

Dig through past orgs and projects—somewhere in the archives lies *your* perfect match. Choose wisely, explorer!

Treasure Hunt: Find Your Open-Source Goldmine!

The GSoC orgs drop! It's like waiting for the new season of your favourite show—but with more code and fewer cliffhangers

Feb 27: The Grand Reveal!

Assemble Your Arsenal: Tech Stack & Git-Fu!

Pick your weapon—Python, JS, C++? Master Git like a ninja (commits, branches, PRs = your secret moves). No Git, no glory!

Enter the Arena: Contribute & Impress!

Time to battle… with code! Fix bugs, open PRs, chat with mentors—but keep it classy. No chaotic commits, only clean code!

March 24 - April 8: Submit or Bust!

The final boss level—submit your proposal before time runs out. No take-backs, no retries, just go for it!

08 Pick Your Tech Stack: Choose Wisely, Code Strong!

1. Tech Stack = Your Superpower!

Choose tools that suit your project's needs and your skill level.
Examples: If you're building web apps, JavaScript (React/Node.js) is your Swiss army knife. For data, Python (pandas, TensorFlow) is your trusty lightsaber.

2. Know Your Project, Know Your Stack!

Different stacks for different missions. Choose based on what your project requires!
For simple websites? HTML, CSS, and JS will do. For back-end heavy apps? Python (Flask/Django), Ruby, or Java might be your best sidekick.

3. Do You? Need Speed or Scalability?

Some stacks are built for speed (Node.js, Go), others for scalability (Java, Spring Boot, PostgreSQL). What's your goal?
Speed = Node.js. Scalability = Java with Spring Boot.

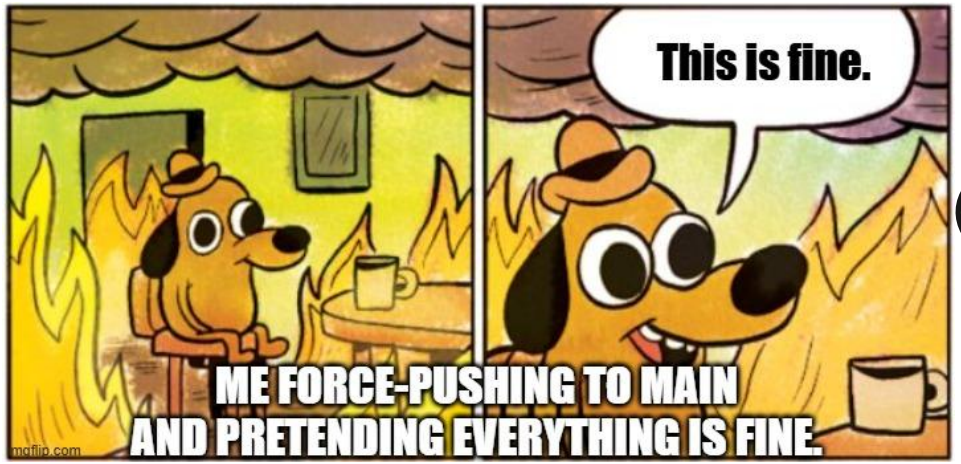


4. Familiarity is Key!

Stick with what you know (unless you want to level up!).
Stuck on JavaScript? Stick with Node.js and Express. Got Python powers? Try Django or Flask.

09 Version Control: *Git Gud* or *Git Lost!*

1. **GitHub, GitLab, Bitbucket**—choose your platform wisely.
2. **Fork, clone, commit, push, pull, merge—key Git actions** for smooth collaboration.
3. **Branch** for features, **merge** for progress, and **avoid committing mistakes.**
4. Messed up? No worries — **git reset, git revert, git blame** *(oops)*



"Think of Git as your time machine—except instead of fixing past mistakes, you just pretend they never happened."

10 Finding Your Tribe - Org Hunting Season!



1. **Dig** and **SCOUR** through past GSoC orgs like a hacker on a mission. **Best Time to start:** *November*

2. **Match** their tech stack with your skills (or level up fast!).

3. **Sneak into their Slack/Discord/mailing lists**—be present, not a ***ghost***.

4. ***Stalk***—I mean, ***read***—project ideas and **start engaging early**!

*"Scouring orgs is less like online dating and more like assembling the **Jedi Council**—pick your team, do some prep work, and get ready to save the code universe!"*

- Master Yoda of the GitHub Galaxy

11 Make Your Mark: The Power of Contributions!

Want to get shortlisted even before GSoC applications open?
Contributions matter! Here's how to build your reputation and stand out.

Start Contributing NOW!

✓ Tackle **easy issues**—bug fixes, small improvements.
✓ Improve **documentation**—orgs love clear, updated docs!
✓ Try to **get PRs merged**—this shows real impact.

Build a Strong Reputation in the Community

1. **Be Early** – Don't wait! Start engaging **before** applications open.
2. **Be Consistent** – Regular contributions > One-time big PRs.
3. **Demonstrate Passion** – Show genuine interest in the project.
4. **Be Proactive** – Don't just ask for tasks—find ways to help!
5. **Be Respectful** – Mentors are busy. **Research before asking questions.**

Why Contributions Matter?

- ✓ The **number of merged PRs** makes a huge impact.
- ✓ Active contributors are often **shortlisted before applications open**.
- ✓ Shows you're **committed & reliable**, not just applying for the stipend.

12 GitHub Etiquette: Code with Clarity, Review with Respect

Commit Smartly:

i. **Keep commits small and meaningful**—Each commit should represent a clear, logical change.
ii. **Use clear commit messages**—Avoid vague messages like "fixed stuff" and instead be specific: "Fix login bug in user authentication."
iii. **Avoid "WIP" commits**—If it's not ready, keep it local until it's complete.
iv. **One task = One commit**—This helps reviewers understand your work more easily.

Effective Communication:

1. **Be clear and specific** with code comments and PR feedback.
2. **Avoid one-word reviews; offer helpful suggestions.**
3. **Don't over-ping mentors**—try to solve minor issues on your own first.

PR Etiquette:

i. **Write clear PR titles**—Your title should describe what the PR is about in a few words.
ii. **Follow the PR template**—Don't skip it! It helps everyone understand the context and intent of your changes.
iii. **Be patient with reviews**—Reviewers might need time. Address feedback carefully and thoughtfully.
iv. **Don't spam PRs**—Make sure your PR is ready for review before submitting it.

13 Proposal Power: Time to Bring Your A-Game!

A GSoC proposal is your way of convincing an organization to accept you as a contributor for the summer. It's super important and here's why:

- **Your First Impression:** It's how organizations get to know you and your skills.
- **Project Plan:** You'll outline what you want to work on and how you plan to do it.
- **Show Your Passion:** It's your chance to show how excited you are about their project.
- **Why You're the Best:** You'll explain why they should choose you over other applicants.



When Should you Start? (Ideally)

Mid-March: Finalize 1-2 orgs and select your project

April 1st - April 8th: Polish, proofread, and submit early

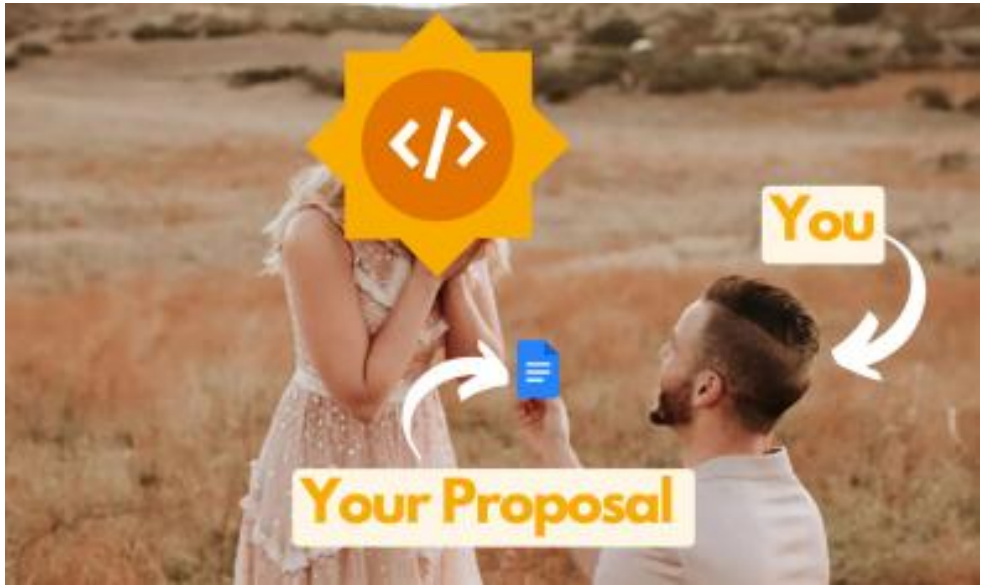
Early March: Create a draft proposal to get a head start

Throughout March: Seek mentor feedback and refine your proposal

If Your Proposal Ain't Solid…

IT'S JUST A CRUSH!

💔



Credits to whoever made the PPT last year

…

14 Proposal Structure and Format

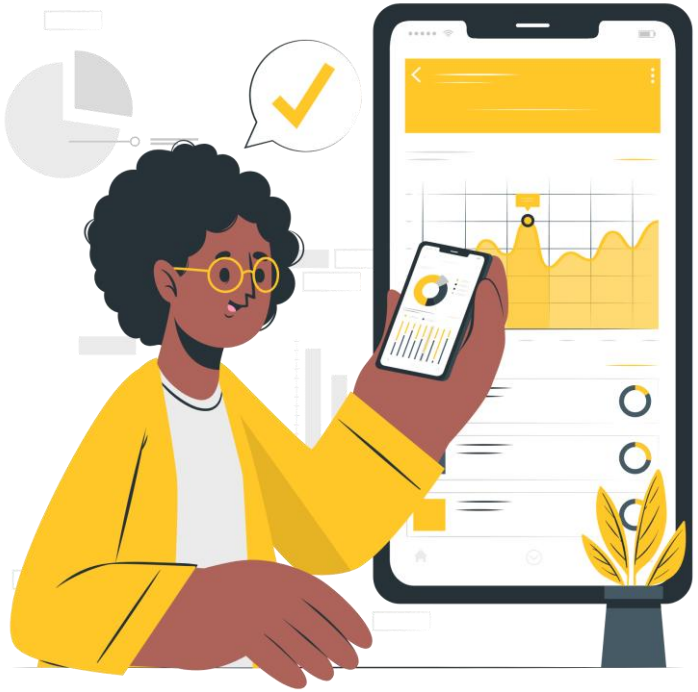
1. Personal Information

Name: [Your Name]
Email: [Your Email]
GitHub: [Your GitHub Link]
LinkedIn: [Your LinkedIn Link]
Relevant Experience:

- Previous open-source contributions (if any).
- Related personal or academic projects.
- Skills relevant to the project.

2. Title & Abstract

- **Title:** Clear, descriptive project title.
- **Abstract:**
- **Problem:** Briefly describe the issue being solved.
- **Impact:** Explain why it benefits the organization
- **Solution:** Outline your proposed approach.



3. Why This Project?

- **Background** on the problem.
- **Importance** of solving it.
- **How it aligns** with the organization's goals.

4. Goals and Features

- Define **success** for the project.
- Key **features** or **improvements** planned.
- **Expected outcomes** and deliverables.

5. Technical Details & Implementation Plan

(Flowchart or Timeline Format)

- **Tech Stack:** List programming languages, frameworks, and tools.
- **Development Phases:**
  - Step-by-step breakdown of work.
  - Milestones and timeline.

6. Mentorship & Communication Plan

- **How you will interact** with mentors.
- **Frequency** and method of **updates.**
- Plan for **progress documentation**.

7. Challenges & Solutions

- **Potential** technical or logistical **issues**.
- **Strategies to overcome** them.

• • •

8. Conclusion & Next Steps

- Summarize project impact.
- What comes next in the application process.
- Call to action (e.g., seeking mentor feedback).